Church Instructions — Release 1

Church-Turing Meta-Machine

Kenneth J Hamer-Hodges

May 2026

Church Instructions Reference

v1.0 — 2026-04-29 CONFIDENTIAL

Overview

The Church instructions implement capability-based access control as hardware-enforced operations. They are the mechanism by which software interacts with Golden Tokens (GTs), capability registers (CRs), and the Namespace. Named after Alonzo Church, these instructions embody the lambda calculus principle of controlled access through abstraction.

All Church instructions that access the namespace route through the **mLoad master validation path**, which enforces permission checks, bounds validation, MAC verification, and G-bit reset on every access.

Architectural Principles

1. **CR0-CR11 Only**: Church instructions can only address CR0-CR11 via the 4-bit register field. Privileged registers CR12-CR15 are physically unreachable through instruction encoding (exception: DREAD/DWRITE may use CR14 as source).
2. **SWITCH Is the Gate**: The only way to write to privileged registers CR12-CR15 is through the privileged SWITCH instruction.
3. **Permission Domains Are Mutually Exclusive**: Turing (R, W, X) and Church (L, S, E) permissions cannot be mixed within a single operation context.
4. **Failsafe FAULT**: Every validation failure (permission, bounds, version, MAC) routes to a FAULT handler — except TPERM, which sets Z=0 on failure to enable conditional execution without trapping.
5. **C-List Mediation**: LOAD and SAVE operate through capability-mediated access, never through raw memory addressing.

6. **mLoad Validation**: All namespace access routes through the mLoad trusted path, which always resets the G-bit on accessed entries.
7. **G-bit Reset on Access**: Every namespace access resets G=0 on the accessed entry, regardless of the instruction or permission type. Reachability determines liveness.

The Ten Church Instructions

1. LOAD — Load Capability from C-List

Purpose: Retrieve a Golden Token from a C-List entry and place it into a capability register.

Required Permission: L (Load) on the C-List capability

Validation Path: Routes through mLoad — permission check, bounds check, GT fetch, namespace validation, MAC check, G-bit reset, thread update.

Operation: 1. Check that the C-List capability has L permission via mLoad → FAULT if not 2. Use the index operand to locate the entry 3. Validate bounds (index must be within valid range) → FAULT if not 4. Validate GT against namespace (version, MAC/seal) → FAULT if invalid 5. Reset G=0 on the accessed namespace entry 6. Copy the capability into the destination CR

Mnemonic: `LOAD CRd, CRs, index`

Aspect	Detail
Permission Check	L on source CR (M-elevation by microcode for CR6)
GT Validation	Version match + MAC seal validation
G-bit Reset	Yes — G bit cleared on namespace access via mLoad
Bounds Check	Index must be within C-List bounds
Result	Validated GT loaded into destination CR

---

2. SAVE — Save Capability to C-List

Purpose: Store a Golden Token from a capability register into a C-List or namespace entry.

Required Permission: S (Save) on the C-List capability

Validation Path: Routes through mSave for write validation; G-bit reset on the accessed C-List namespace entry.

Operation: 1. Check that the C-List capability has S permission → FAULT if not 2. Use the index operand to locate the target entry 3. Validate bounds → FAULT if not 4. Copy the capability from the source CR into the target entry 5. Reset G=0 on the accessed C-List namespace entry

Mnemonic: `SAVE CRd, CRs, index` — CRd is the destination c-list (requires S permission), CRs is the source GT (requires B=1)

Aspect	Detail
Permission Check	S on CRd (destination c-list); B=1 on CRs (source GT)
Bounds Check	Index must be within C-List bounds
Seal Recompute	MAC seal recomputed from Location+Limit
G-bit Reset	Yes — on accessed C-List namespace entry
Result	Capability stored into target C-List entry

---

3. CALL — Protected Call Through Capability

Purpose: Invoke a protected abstraction (service/function) referenced by a Golden Token, saving context for later return. The `imm15` field carries the **method index** for hardware method-table dispatch.

Validation Path: Routes through mLoad for loading the callee’s context — permission check, namespace validation, MAC check, G-bit reset.

Operation: 1. Check E permission on the target CR via mLoad → FAULT if not 2. Validate the GT → FAULT if invalid 3. Reset G=0 on the callee’s namespace entry 4. FAULT if call stack full (256 frames) 5. Push 2-word frame: [caller’s E-GT | NIA+machine_indicators] 6. Set CR6 = callee c-list (E-only), CR14 = callee code (RX, privileged) 7. Hardware method-table dispatch: if imm15 = 0 → NIA = lump_base + 4 (word 1); if imm15 = n > 0 → read memory[lump_base + n×4]; zero entry → PRIVATE_METHOD FAULT; else NIA = lump_base + entry×4

Mnemonic: `CALL CRs [, #index]`

Aspect	Detail
Required Permission	E (Enter) on source CR
GT Validation	Version match + MAC seal validation
G-bit Reset	Yes — on callee namespace entry via mLoad
Frame Pushed	2 words: [caller E-GT | NIA+machine_indicators] — no CRs or DRs saved
Stack Overflow	FAULT if stack full (256 frames). stackSpace indicator updated
Method index = 0	NIA = lump_base + 4 (word 1, single entry point, backward-compatible)
Method index n > 0	NIA = memory[lump_base + n×4]; zero entry → PRIVATE_METHOD FAULT
PC = 0	Always FAULTs — lump header (word 0) is never a valid entry point
New Context	CR6 = callee c-list (E-only), CR14 = callee code (RX, privileged)
Unchanged	DR0–DR15, CR0–CR5, CR7–CR13, CR15 — callee inherits all from caller

---

4. RETURN — Return from Protected Call

Purpose: Restore context saved by a previous CALL and resume execution at the caller.

Required Permission: None (return is always permitted if a saved context exists)

Frame structure: CALL pushes 2 words. RETURN pops them. The caller's E-GT in Word 0 is revalidated by mLoad — this catches use-after-free (version mismatch → FAULT) and resets the G-bit for GC liveness tracking.

Operation: 1. Check that a frame exists on the call stack → FAULT if empty 2. Pop Word 0 (caller's E-GT) and Word 1 (NIA | machine indicators) 3. mLoad the caller's E-GT: version check + MAC + G-bit reset → FAULT on failure 4. Re-run NS split on caller's NS entry → re-derive CR6 (caller c-list) and CR14 (caller code) 5. Restore PC from NIA (Word 1); restore machine indicators from Word 1 6. Apply mask[11:0]: all CRs with mask bit set written to NULL in one parallel clock edge

Mnemonic: `RETURN [mask]` — mask is a 12-bit literal in instruction bits [11:0]; mask=0 is the no-op default (`RETURN` with no argument)

Aspect	Detail
Permission Check	None
E-GT Revalidation	mLoad on caller's E-GT: version, MAC, G-bit reset (FAULT on failure)
CR6 / CR14 Restore	Re-derived from caller's NS entry via NS split — not stored directly in frame
CR5	Thread register — installed by CHANGE from Zone ④ bounds; not touched by CALL/ RETURN
PC Restore	NIA from Word 1
Machine Indicators	Restored from Word 1 (LAMBDA-active, flags, stackSpace, etc.)
Mask	bits [11:0] — **not implemented in current hardware**; all bits ignored. Bit 6 reserved (must be 0 — CR6 always re-derived from E-GT). Assembler warns if any bit is set. Use bare `RETURN` (mask=0).
Unchanged	DR0–DR15 and non-masked CRs retain callee values
Stack Underflow	FAULT: no saved context
Stack Indicators	stackFrames and stackSpace updated

Mask field status: The mask field is reserved for future implementation. It is not enforced by current hardware — encoding a non-zero mask produces an assembler warning but the clearing does not occur. Always use bare `RETURN` (mask=0) in current programs.

---

5. CHANGE — Change Thread Context

Purpose: Suspend the current thread and activate another — full atomic context swap. CHANGE indexes a Thread Abstraction GT from a C-List and performs the complete per-thread state exchange.

Validation Path: Indexes CRd at offset idx; verifies E permission on the Thread Abstraction GT.

Operation: 1. Index CRd at offset idx; verify the GT has E permission — FAULT on fail 2. Suspend outgoing thread: save per-thread state (DR0–DR15, CR0–CR11, STO, PC, FLAGS) into its thread lump; CR13, CR14, CR15 are never touched 3. Activate incoming thread: restore per-thread state from its thread lump 4. The suspended thread resumes exactly where it left off

Mnemonic: `CHANGE CRd, idx`

Aspect	Detail
Permission Check	E (Enter) on Thread Abstraction GT at CRd[idx]
Per-Thread Saved/Restored	DR0–DR15, CR0–CR11, CR14 (code register), CR15 (namespace root), STO, PC, FLAGS
System-Wide Unchanged	CR12 (thread stack), CR13 (IRQ handler)
G-bit Reset	Yes — on accessed namespace entries

---

6. SWITCH — Install PassKey into System Register

Purpose: Write a PassKey capability into system register CR13 (IRQ Thread) or CR15 (Namespace root). This is the only instruction that can modify these system-wide registers, making it the privilege gate for namespace and interrupt management. CR12 (thread stack) and CR14 (transient code-view) cannot be SWITCH targets.

Validation Path: Two mandatory hardware checks before the write; namespace access via mLoad with G-bit reset.

Operation: 1. Decode the 3-bit `Tgt` field; FAULT if Tgt is not CR13 (101_2) or CR15 (111_2) 2. Check source register is CR0–CR11 (4-bit encoding); FAULT if out of range 3. **PassKey type check** — CRs.word0_gt.gt_type must equal 11_2 (Abstract GT); FAULT if not 4. **Sentinel address check** — CRs.word1_location must equal the reserved hardware sentinel for the chosen target; FAULT on mismatch: - Tgt = CR13: sentinel = `0xFFFFFFFE` (all-1s − 1) - Tgt = CR15: sentinel = `0xFFFFFFFF` (all-1s) 5. Install CRs into the target system register via mLoad; reset G=0 on accessed namespace entries

Mnemonic: `SWITCH CRs, target`

Aspect	Detail
Valid Targets	CR13 (Tgt=101_2, IRQ Thread), CR15 (Tgt=111_2, Namespace) — all others FAULT
PassKey type	CRs.word0_gt.gt_type must be Abstract (11_2); any other type → FAULT
Sentinel for CR13	CRs.word1_location == `0xFFFFFFFE` (all-1s − 1)
Sentinel for CR15	CRs.word1_location == `0xFFFFFFFF` (all-1s)
Sentinel mismatch	CRs sentinel does not match chosen target → FAULT
G-bit Reset	Yes — via mLoad on namespace access
Not writable by SWITCH	CR12 (thread stack, system-wide), CR14 (transient cLoad output)

TPERM — Test Permission / GT Health Check

TPERM is the single-instruction GT health check. It evaluates permissions, validity, and bounds in one cycle and **sets condition flags** — it does not trap. The flags persist across subsequent instructions, enabling ARM-style conditional execution for zero-cost try-catch patterns.

```
TPERM CRs, #preset [, offset]
```

What TPERM checks (all at once): 1. **Permissions** — does the GT have the requested permission bits? (R, W, RW, E, LS, etc.) 2. **Valid** — does the GT pass version and MAC validation? 3. **Base + Limit** — if an offset is provided, is Base + offset within the GT’s region?

Flags set: - **Z = 1**: all checks passed (permissions present, valid, in bounds) - **Z = 0**: one or more checks failed

Faulting: TPERM faults with `TPERM_RSV` if the preset code is reserved (codes 11–15 and their B-modifier variants 0x1B–0x1F). For all valid presets, TPERM does not fault — if a permission check fails the Z flag says so and software decides what to do via conditional execution. The actual LOAD/SAVE/CALL instructions that follow enforce safety. TPERM is the “ask first” instruction.

No namespace access occurs — TPERM is a register-local read-only operation. No G-bit reset.

Conditional Execution: Zero-Cost Try-Catch

Because every Church Machine instruction carries a 4-bit ARM condition code, TPERM + conditional suffixes give you try-catch with no branches and no overhead on the happy path:

```
TPERM CR5, RW, offset      ; check R+W perms, valid, base+offset in bounds
                           ; Z=1 if all pass, Z=0 if any fail

readEQ DR1, CR5, offset     ; happy path — only fires if Z=1
IADDEQ DR2, DR1, 1          ; happy path — continues if Z=1
writeEQ CR5, offset, DR2    ; happy path — writes if Z=1

; catch path (TBD — recovery is case-by-case)
; instructions with NE suffix fire when Z=0
```

The happy path does not branch, does not check errors, does not even know failure is possible. Every instruction carries EQ and the hardware silently skips it if TPERM failed.

Permission Restriction (monotonic)

TPERM can also restrict permissions on a GT:

```
TPERM CRd, #preset
```

The assembler encodes restriction mode by setting `imm15 = 0x7FFF` (all fifteen bits set). This sentinel distinguishes restriction from a health-check at offset 0 (`imm15 = 0`), which is a valid bounds test of the base address itself. The full range 0–32766 is available for health-check offsets.

Permissions can only be removed, never added (monotonic restriction). Domain purity is enforced: Turing (R, W, X) and Church (L, S, E) permissions cannot be mixed.

Full Metadata Query

TPERM can read all metadata fields from a CR into a data register:

Result Layout (DRd): | Bits | Field | Description | |——|——-|———-| | [5:0] | Permissions | R, W, X, L, S, E from CRs Golden Token | | [10:6] | (Reserved) | Zero | | [11] | stackFrames | 1 = at least one frame on call stack (RETURN safe) | | [12] | stackSpace | 1 = room for at least one more frame (CALL safe) | | [13] | Valid | 1 = GT passes version and MAC validation | | [15:14] | Type | GT type: 00=NULL, 01=Inform, 10=Outform, 11=Abstract | | [31:16] | (Reserved) | Zero |

Fused Church Instructions

7. LAMBDA — In-Scope Code Application

Applies the code object referenced by a CR in the current scope. Requires X permission. Does not switch c-lists. See `docs/lambda-instruction.md` for full specification.

8. ELOADCALL — Fused Load + Call

Loads a GT from a c-list row (word offset) and immediately enters it. Atomic: no intermediate CR state is visible. imm15 is split: bits[7:0] = c-list row (0–255), bits[14:8] = method index for the CALL phase (0–127). Equivalent to `LOAD CRd, CRsrc, #row` followed by `CALL CRd, #method_index`, but atomic.

9. XLOADLAMBDA — Fused Load + Lambda

Loads a GT from a c-list row (word offset) and immediately applies it. Equivalent to `LOAD CRd, CRsrc, #row` followed by `LAMBDA CRd`, but atomic.

Encoding

Instruction Encoding Format

```
31    27 26  23 22  19 18  15 14           0
|opcode | cond |  dst |  src |    imm15    |
| 5 bit | 4 bit| 4 bit| 4 bit|   15 bits   |
```

Every instruction can be conditionally executed based on the condition flags. This is what enables the TPERM try-catch pattern — one TPERM sets the flags, subsequent instructions carry EQ/NE suffixes.

Security Model Summary

The Church Machine enforces these core security invariants:

Principle	Enforcement
No direct privileged register access	CR0-CR11 only in instruction encoding (4-bit field)
Privilege through SWITCH only	Only SWITCH writes to CR12-CR15
Permission before access	Every Church instruction checks permissions before operating
Single validation path	All namespace access routes through mLoad
G-bit reset on every access	mLoad always resets G=0, ensuring GC accuracy
Failure handling	All violations route to FAULT handler
Capability-mediated access	LOAD/SAVE go through C-List capabilities, never raw memory
Mutually exclusive domains	Turing (R,W,X) / Church (L,S,E)

Confidential — Kenneth Hamer-Hodges — April 2026